

Building with Coding Agents

Ship a Python Streamlit Dashboard

PyLadies Amsterdam · July 14, 2026



Hi, I'm Felicity

Data scientist · applied ML & MLOps

I build automated workflows, AI agents, and end-to-end pipelines

Route here: coding-agent projects + hackathons

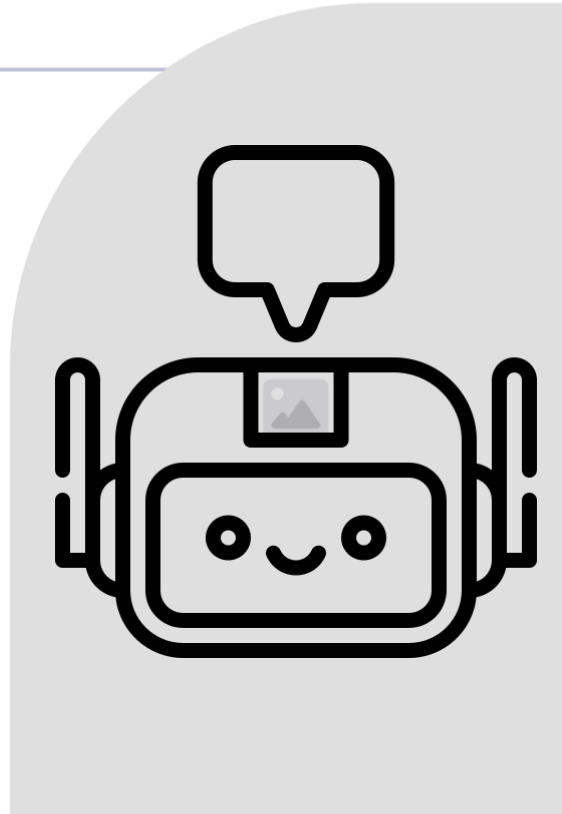
Tonight: you direct, the agent codes.

What is a coding agent?

A language model connected to your computer: it can read your files, run commands, write code, and see the results — then decide what to do next.

Chat AI

Answers questions
and explains
concepts.



Coding Agent

Reads files, runs
commands, writes
and tests code.

Examples: Claude Code, Gemini CLI, Cursor.

You are the architect. The agent is the contractor.

Don't micromanage every line. Don't say "build me a house" and disappear.

Write a brief a skilled professional could execute.

Context

What this is and who it's for

Requirements

Exactly what to build

Constraints

What not to do

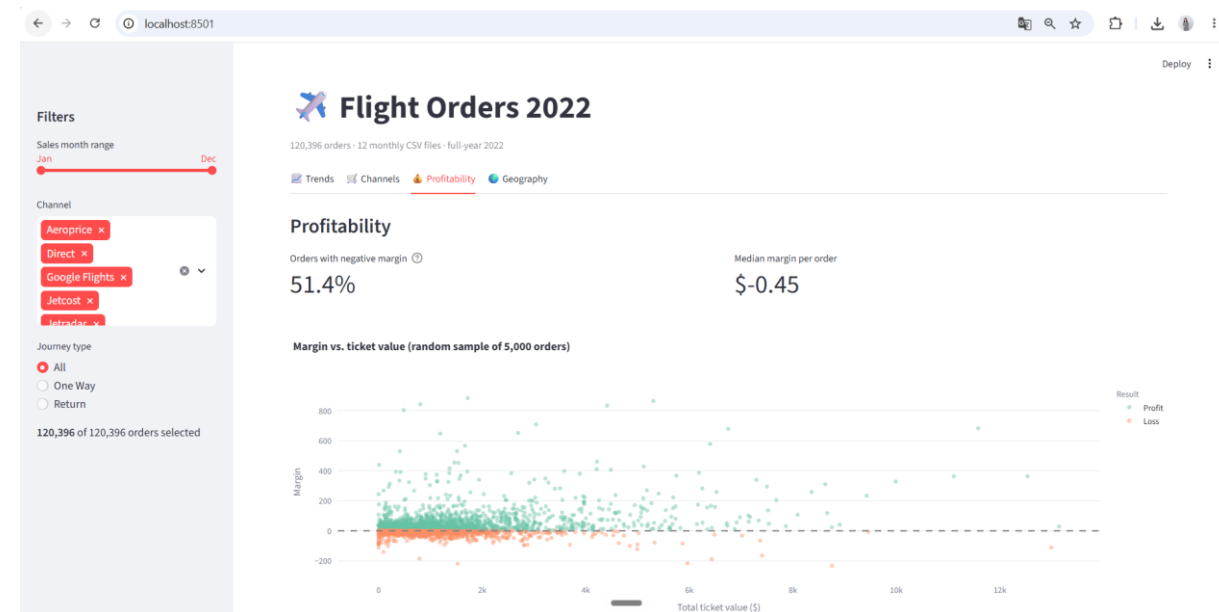
Checkpoint

"Show me the plan first"

What you'll build tonight

A real dashboard from **120,396 flight orders** — 4 KPI views, 3 filters — almost entirely by directing an agent.

 **A fact hiding in the data:** more than half of all orders lose money (median margin: $-\$0.45$).



Tools tonight

Claude Code

My demos — needs a Claude subscription or API key

Gemini CLI

Free with a Google account

Any agent

The exercises are prompts and markdown, not code

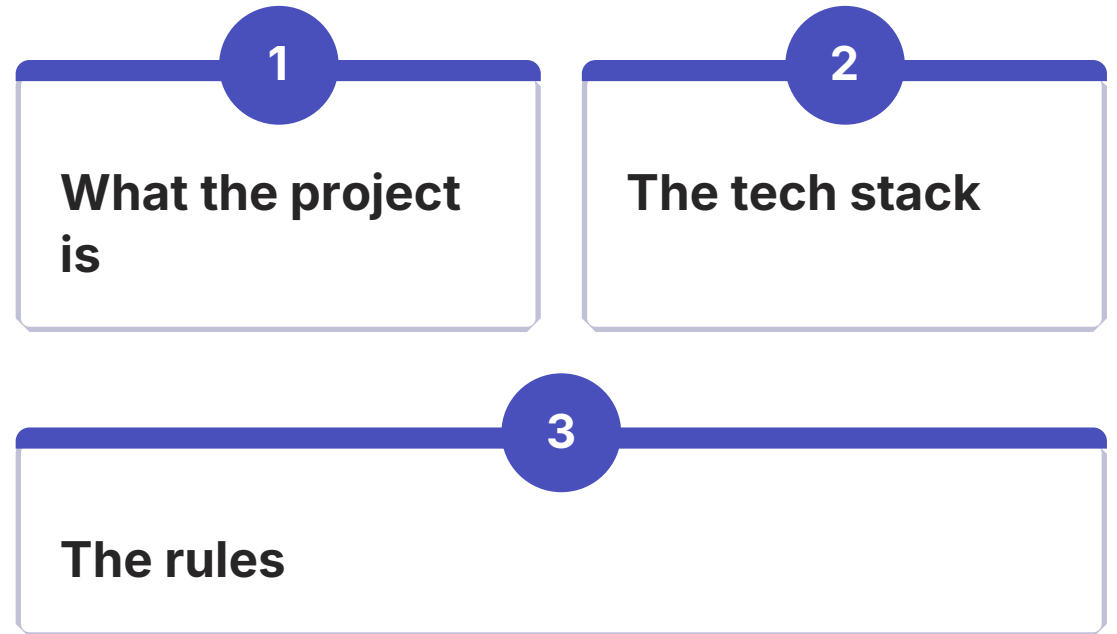
Smoke test — paste in your agent:

"Read data/DATA.md and tell me how many CSV files this project has" → the answer is **12**.

CLAUDE.md — the agent's onboarding document

Every session starts with **total amnesia**. CLAUDE.md is the file the agent reads first, every time: what the project is, the tech stack, the rules.

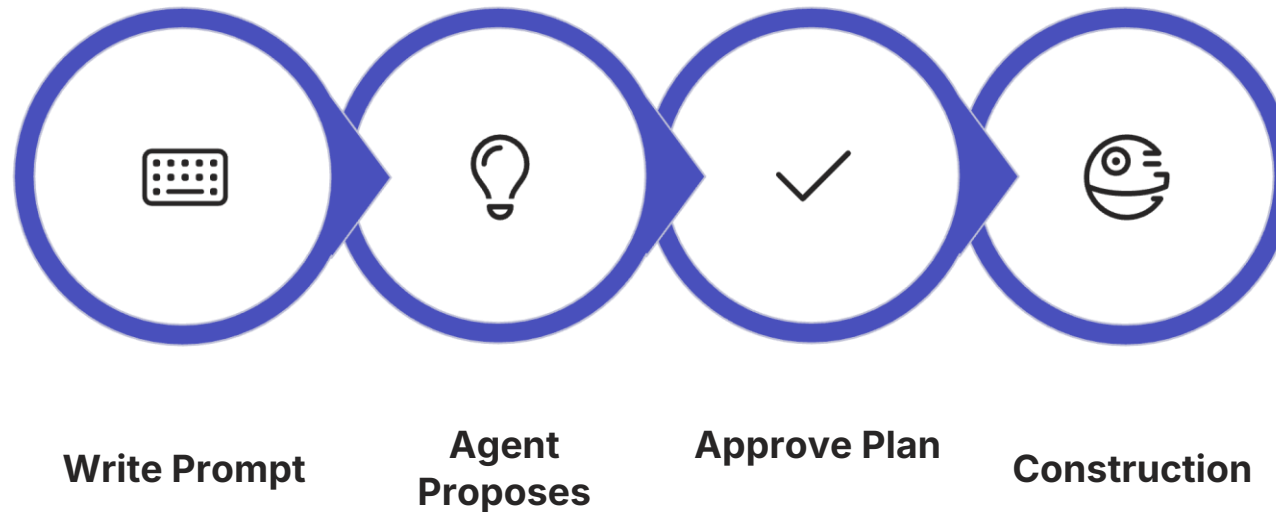
Write it once — stop re-explaining forever.



Keep it lean: it is loaded always, so every line costs.

Plan before code

"Propose your plan first. Wait for my approval."



Review the architectural drawings before construction starts. The plan is where a mistake costs **one sentence to fix** — instead of a rebuild.

Guardrails — the polite kind

Rules you state in words — the agent reads them and **usually** honors them.

Remember that "usually". We fix it in Block 2.

data/ is read-only

The agent must never
modify source data files

**Ask before adding
dependencies**

No surprise packages in
your environment

■ EXERCISE 1

🔧 Exercise 1 — The Brief & the Plan

Files — in order

1. `starter-prompt.md`
2. `CLAUDE-template.md`
3. `requirements-card.md`

Done = your agent proposed a plan, you approved it, dashboard v1 is building.

Where to start

Open `workshop.html` → Exercise 1

Stuck?

Open `solutions.html` — copy, continue, study later.



The exercises are prompts and markdown — any agent works.

UP NEXT

Skills, Hooks & Multi-Agent Patterns

A practical guide to expertise written down, guardrails with teeth, and knowing when one agent isn't enough.

SKILLS

Markdown files of instructions the agent absorbs as knowledge

SLASH COMMANDS

Buttons you press — triggered by you, by typing `/name`

HOOKS

Guardrails enforced by the system — cannot be forgotten or talked past

Skills & Slash Commands — Expertise, Written Down

Both are markdown files of instructions. The difference is who pulls the trigger:

	Skill	Slash Command
Triggered by	The agent, automatically, when relevant	You, by typing <code>/name</code>
Mental model	Knowledge it absorbed	A button you press
Example	"here's how we style charts"	<code>/check-dashboard</code>

 Skills load only when needed — cheap. **Rule of thumb:** the third time you type the same prompt, save it.

The Context Window Is a Budget

Everything the agent "knows" right now lives in a limited window — and quality degrades as it fills (*context rot*).

CLAUDE.md = always-paid tax

Keep it lean. Every token here is spent on every single request.

Skills = pay-per-use

Load only when relevant. The cost is zero until the moment they're needed.

Long sessions = compact & clear

Run `/compact` at milestones, `/clear` between tasks. Files survive compaction; conversation fades.

Files survive compaction; conversation fades.

The Community Builds With These Exact Primitives

One compresses what the agent says; the other what it reads.

Together they attack context rot from both directions — output verbosity and input bloat.

caveman

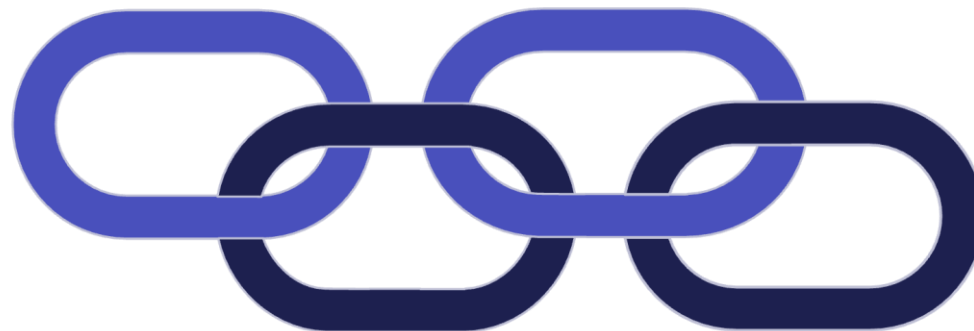
A skill that makes agents talk terse: ~65–75% fewer output tokens.

rtk

A hook that compresses tool outputs 60–90% before they reach the window.

Caveman Output

RTK Tool Input



65–75% Fewer

60–90% Smaller

I use both.

ENFORCEMENT

Hooks — The Guardrails With Teeth

A hook runs automatically on events (before a file edit, after a command) — enforced by the system, not by the agent's goodwill.

It cannot be forgotten, ignored, or talked past.

Soft rule

"please don't touch
data/"

Hard rule

The edit is **blocked**
before it executes.

CLAUDE.md is the sign that says *please wash your hands*.

A hook is the door that won't open until you do.

■ EXERCISE 2

🔧 Exercise 2 — Standards & Limits



Open `workshop.html`

Navigate to Exercise 2



Use `SKILL-template.md`

One vague prompt → `hook-template/`
→ try to break it (two rounds)



Done when...

Your dashboard is restyled to **your** rules, and you watched a write get blocked.



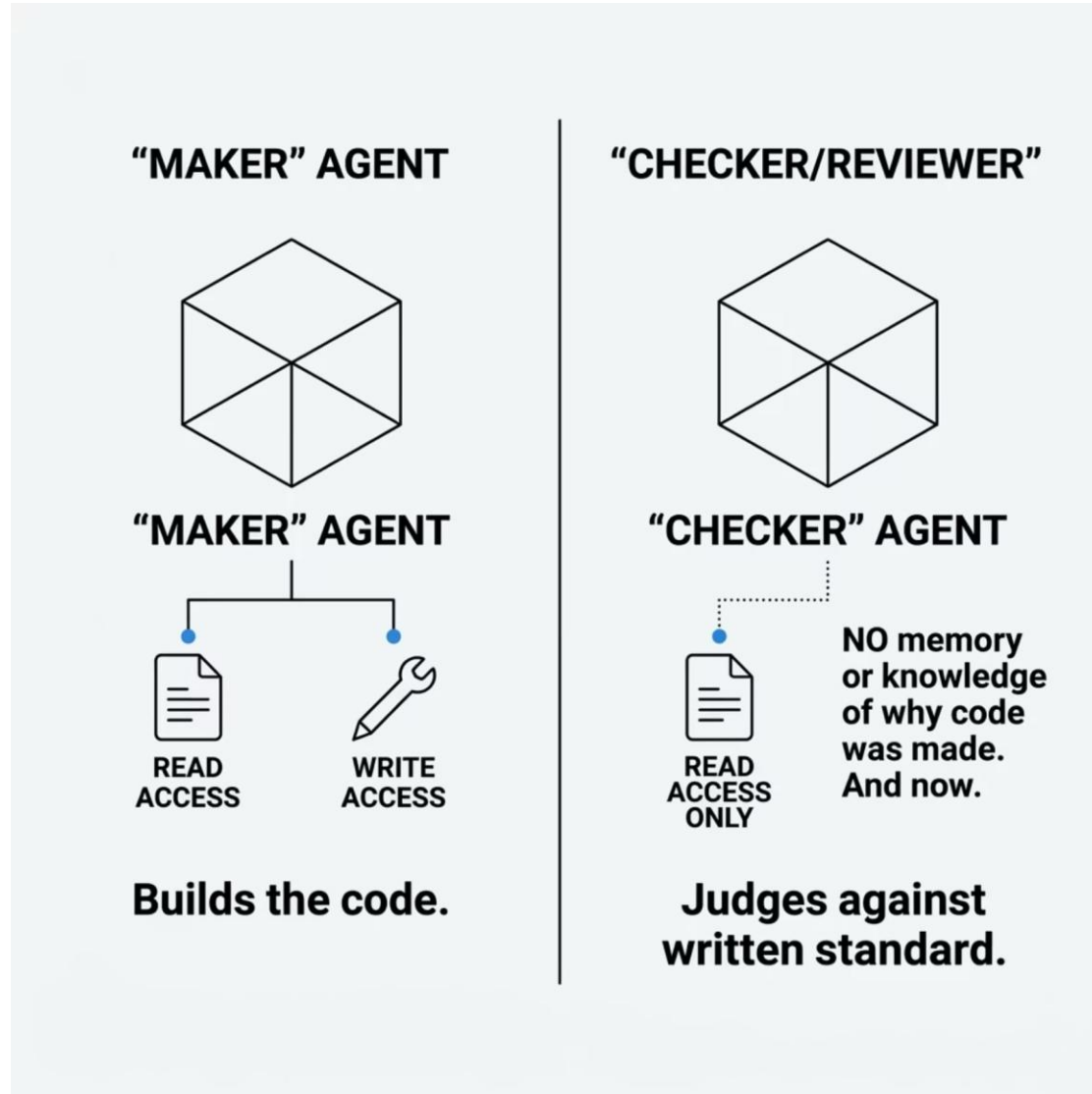
Remember: new session after installing — settings load at session start.

One Agent, or Many?

Add a second agent when you can name what the first one can't do — not before.

	One Agent	Parallel Subagents	Specialist Roles
When	Sequential steps, shared context — most tasks (the default!)	Independent pieces, like our 4 view files	Different jobs need different tools
Wins	Continuity, consistency	Wall-clock ÷ 4, small fresh context each	Fresh eyes, honest checking

Never Let the Maker Be the Checker



A reviewer agent with only **Read tools** — it literally cannot edit.

Fresh eyes, no memory of why the code is that way, judging only against your written standard.

| Constraint by architecture beats constraint by politeness.

■ EXERCISE 3

🔧 Exercise 3 — Delegation



Part A

Make your agent argue both sides



Part B

4 parallel subagents, one per view —
overlapping timers, batch finishes in the
slowest one's time



Part C

`reviewer-template.md` → the audit



Done = a violations table from an agent that fixed nothing. Watch the screen for the parallel launch.

MCP — giving the agent hands (and eyes)

Skills teach

The agent learns *how*: "here is how we style charts"

MCP connects

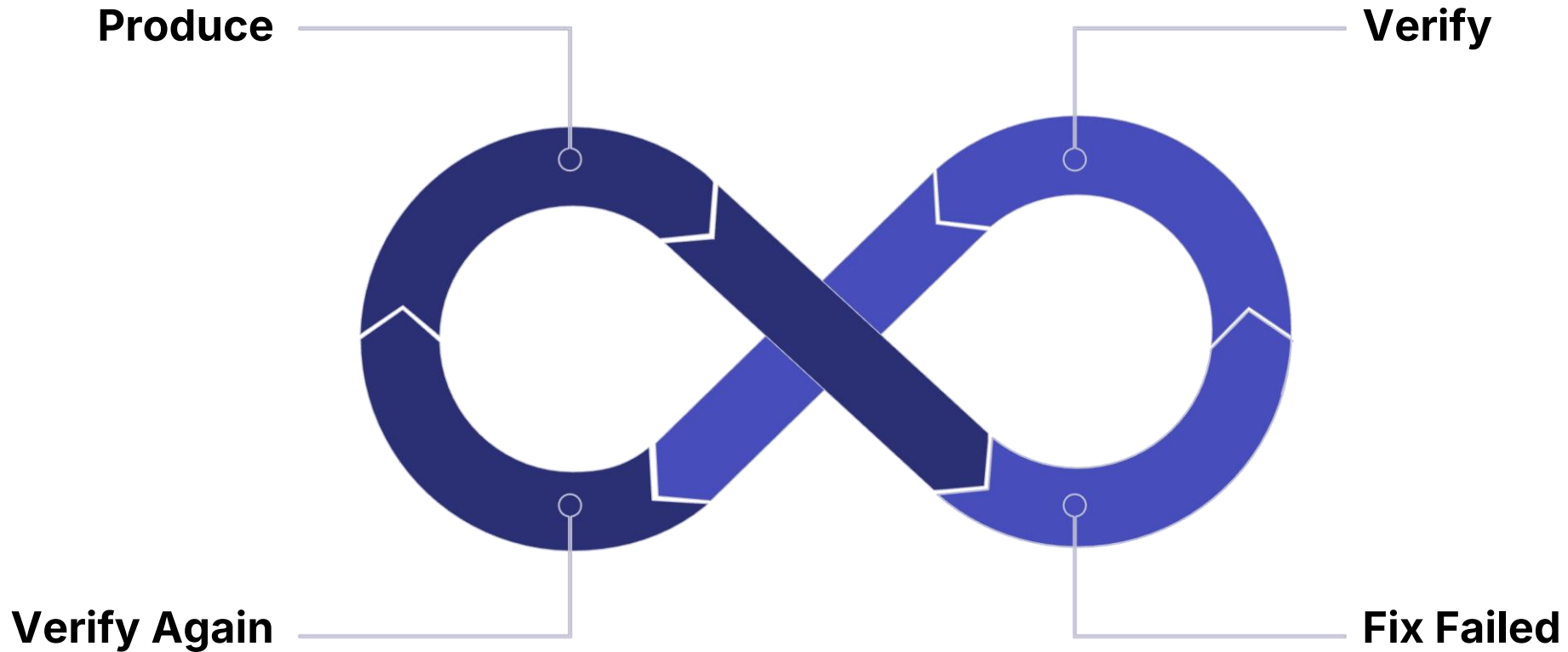
The agent gains *access*:
browsers, databases,
Notion, GitHub — called by
name.

Model Context Protocol — the USB-C of AI

One standard plug between any agent and any tool.
A server exposes a menu of tools; the agent calls them by name.

📄 Tonight's server: **Playwright** — a real browser the agent can navigate, click, and read.

The loop: how agents become reliable



One attempt

~80% right — good, but not reliable enough for production.

The catch

The agent rises exactly to the standard of your verifier — **never above it.**

A loop with checks

Converges toward reliable. Each cycle catches what slipped through.

Always set a retry cap

A loop without a stop condition is **a bill without a ceiling.**

■ EXERCISE 4

Self-Verification (demo-first)

What you'll watch

verify → sabotage → the loop catches it → fix → **PASS**
Then saved as `/check-dashboard` — one word, forever.

Following live

Requires **Playwright MCP** set up — a 150 MB download.

- **Tonight in the room:** watch the demo
- **Tonight at home:** do it yourself

Evals — is any of this actually working?

Verifier

Judges *the work*: "Is this change ok?"

Eval

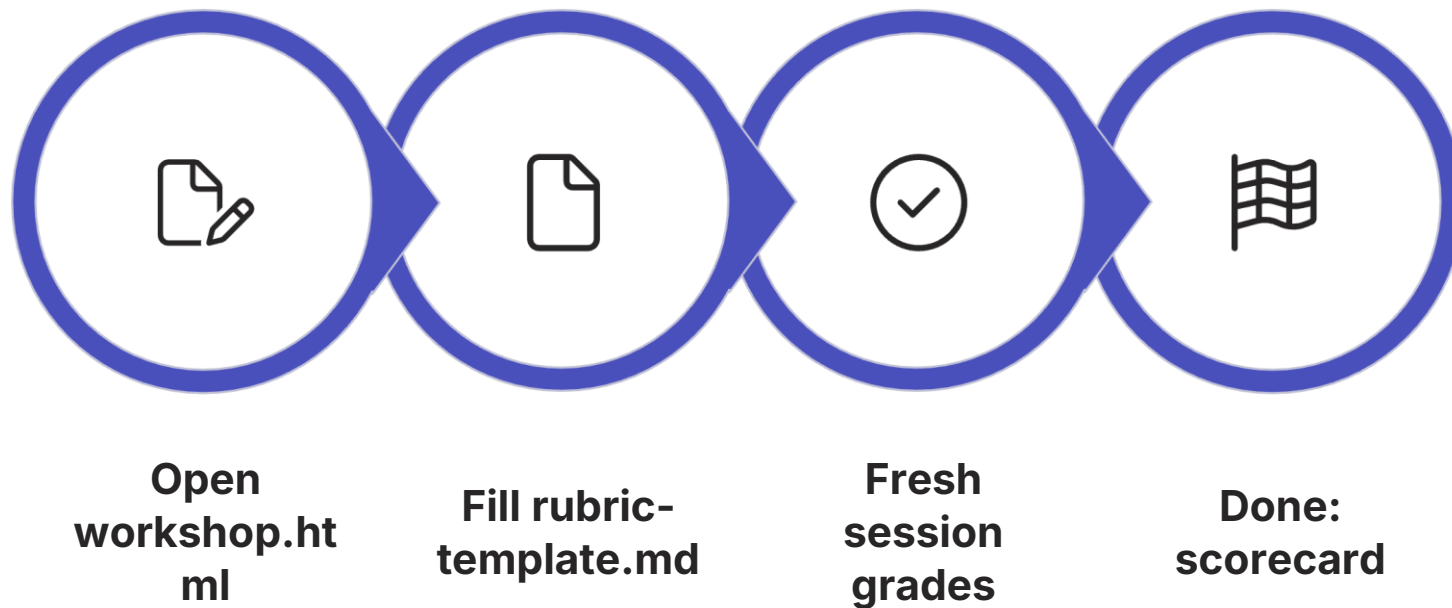
Judges *the worker*: "Is my whole setup good — and improving?"

A rubric = your promises, made gradeable

Checkable criteria + evidence + verdicts.

- Include one criterion that **FAILS on purpose** — a rubric that only measures what you already built is a mirror, not an instrument.

Evaluation (the finale)



The template

rubric-template.md: 3 given + 4 yours +
1 aspirational + 1 judgment

The rule

A fresh session grades your dashboard —
and **fixes nothing**.

Done means

Your scorecard: everything promised
delivered, next milestone named.

Agents forget everything overnight. Leave them a note.

	README	CLAUDE.md	MEMORY.md
Is	The brochure	The handbook	The shift-handover note
For	Humans	Every agent session	The <i>next</i> session
Changes	Rarely	Occasionally	Every session

 Tonight's homework, one prompt: **"Summarize this session into MEMORY.md."**

What you actually learned tonight

01

The verifier loop = the mechanism

The core primitive that makes agents reliable.

03

Long-running agents = that design, deployed unattended

The natural endpoint of everything you built tonight.

02

Loop engineering = designing the system around it

Goal, verifier, memory, termination. *The term is a month old
— you'll see it on your feeds.*

04

Prompt → context → loop engineering

You practiced all three tonight.

How far do you let it run?

1

1. Watch every step

2

2. Auto-approve safe actions

3

3. Task-level autonomy

"Show me at the end"

4

4. Background agents

Overnight runs, PRs in the morning

Where the value is today

Most professional value today lives at **rungs 2–3**.

Each rung up needs the guardrails from the rung below.
You built those tonight.



An agent can only run as long as your guardrails can hold.

Thank you

[workshop.html · solutions.html · github.com/pyladiesams/building-with-coding-agents-jul2026](#)

Recording on YouTube.

This workshop was built with the methods it teaches — every exercise dry-run by an agent, every failure folded back into the materials.

@pyladiesams · @Fan-shiyu

